

SECURITY & COMPLIANCE

Security & compliance, by design.

How FinFlo protects tenants and their members — two separate identity systems, strict per-tenant isolation, role-based authorization, hardened sessions, encrypted personal data, and a complete audit trail.

DOCUMENT

Security & Compliance

AUDIENCE

Security · Compliance

VERSION

1.0

PRECISION ENGINEERING FOR MODERN FINANCE

IDENTITY

Two separate identity systems

FinFlo runs two completely distinct identity systems so the business side and the customer side never share a session. **Users** are the business side — `super_admin`, `admin`, and `staff` — authenticated by the `protect` middleware under `/api/auth`. **Members** are the end-customers of the member portal, authenticated by `protectMember` under `/api/member-auth`, where `req.member.user` resolves to the owning business.

Every JWT carries a `type` claim, and each middleware rejects tokens of the wrong type — a member token can never reach a business route, and vice-versa. Tokens travel by `Authorization: Bearer` header on mobile and by the `token` cookie on web.

TOKEN TYPE	ISSUED TO / FOR	ACCEPTED BY
<code>user</code>	Business side — <code>super_admin</code> / <code>admin</code> / <code>staff</code>	<code>protect</code> (routes under <code>/api/auth</code>)
<code>member</code>	End-customer of the member portal	<code>protectMember</code> (routes under <code>/api/member-auth</code>)
<code>pin</code>	Transaction-PIN verification	PIN-gated member flows
<code>2fa-pending</code>	Login awaiting TOTP second factor	The 2FA challenge step only

Why it matters

Separate middleware plus a typed token means a stolen or replayed credential is useless outside its own surface — business and customer authentication never cross.

ISOLATION

Multi-tenant isolation

Every business that signs up is an isolated **tenant**. Tenant data never mixes because every tenant-scoped query is scoped by the **effective owner**, not by the requesting user.



Effective owner scoping

The auth layer sets `req.user.effectiveOwnerId` — an admin's own id, or a staff member's `ownerId`. Tenant queries scope by it, never by `req.user._id`.



Branch narrowing

Branch managers are narrowed further by `managedBranchId` (cached on the User doc) so they see only their branch's data.



404, never 403

An unowned resource returns **404**, not 403 — the platform never leaks the existence of another tenant's records.



Cascading attribution

Branch and owner changes cascade to every dependent collection so per-tenant and per-branch analytics stay correct.

AUTHORIZATION

Role-based access control

Authorization is driven by `Role` documents, each carrying a `permissions` array. The platform resolves a user's permissions and gates routes against them, so access is data-driven rather than hardcoded.

Permission gates

Routes are gated with

```
authorizePermissions('manage_x',  
...)
```

 — a request without the named permission is rejected.

Wildcard & bypass

The `'*'` wildcard and `super_admin` bypass all permission checks for platform operators.

Coarser helpers

The `admin` and `staffOrAdmin` helpers cover broad role gates where fine-grained permissions aren't needed.

SESSIONS

Session & auth hardening

Sessions are built to limit the blast radius of a compromised credential and to be revocable in real time.



Revocable sessions

Sessions can be revoked server-side; a revoked session is rejected on its next request and pushed off live sockets.



Short-lived web tokens

Web access tokens are short-lived (≈15 minutes) and native-safe, and are kept out of `localStorage` to reduce token theft.



Step-up re-auth

High-risk actions require step-up re-authentication, so an active session alone isn't enough for sensitive operations.



2FA & logout

TOTP-based two-factor authentication and brute-force logout protect the login surface; member sessions have parity with the same model.

CREDENTIALS

Password policy

Every password must be at least **8 characters** and include at least **one uppercase letter**, **one number**, and **one special character**. The policy is enforced **server-side on every password set, change, and reset endpoint** — the server is the source of truth.

Min 8 characters

1 uppercase

1 number

1 special character

Client experience

The client surfaces the policy as a hint and shows a real-time strength meter as the user types — guidance on the front end, enforcement on the back end.

DATA PROTECTION

Protecting personal data

Sensitive member data is encrypted at rest, and the API surface is hardened against common injection and tampering vectors.



Field-level encryption

Sensitive Member fields are encrypted at rest via `encryptFields` / `decryptFields`; the CNIC is stored encrypted with a hash for searchable per-tenant uniqueness.



Injection sanitization

Input is sanitized against NoSQL operator injection — `$` and `.` keys are stripped from body, params, and query — and HPP protection blocks parameter pollution.



Transport & headers

`helmet` sets security headers and CORS is restricted; per-path rate limiting (auth / otp / signup / generic) throttles abuse.



Fail-closed startup

The server hard-fails on startup if `JWT_SECRET` is weak or short, or if `MONGO_URI` is missing — it refuses to run misconfigured.

COMPLIANCE & AUDIT

Auditable by construction

FinFlo gives regulated lenders the controls and capabilities they need to meet KYC and AML obligations and to reconstruct any money movement after the fact.



KYC document vault

Member documents are held with explicit verification states (Pending / Verified / Rejected / Expired) so staff can confirm identity before activation.



AML / CTR monitoring

Anti-money-laundering and currency-transaction-report monitoring is built into the compliance pillar.



Full audit trail

The `FinancialTransaction` ledger records auditable money movement and `ActivityLog` records user actions — together a complete trail.



Expiry reminders

Document-expiry reminder jobs flag KYC documents before they lapse so records stay current.

Capabilities, not certifications

This document describes the security controls and compliance capabilities FinFlo provides. It does not assert any formal compliance certification or audit result.

Security controls summary

CONTROL AREA	HOW FINFLO HANDLES IT
Identity	Two separate systems (Users / Members) with typed JWTs; each middleware rejects wrong-type tokens.
Tenant isolation	All queries scoped by <code>effectiveOwnerId</code> ; branch managers narrowed by <code>managedBranchId</code> ; unowned → 404.
Authorization	Permission-array <code>Role</code> docs; <code>authorizePermissions</code> ; <code>'*'</code> / <code>super_admin</code> bypass; admin / staffOrAdmin helpers.
Sessions	Revocable sessions, ≈15-minute web tokens out of localStorage, socket revocation, step-up re-auth.
Second factor	TOTP 2FA with a <code>2fa-pending</code> challenge token; brute-force lockout on login.
Passwords	8+ chars · upper · number · special, enforced server-side on every set / change / reset.
Data at rest	Encrypted sensitive Member fields; CNIC encrypted with a per-tenant uniqueness hash.
API hardening	NoSQL-injection sanitization, HPP, helmet, restricted CORS, per-path rate limiting.
Configuration	Startup hard-fails on weak / short <code>JWT_SECRET</code> or missing <code>MONGO_URI</code> .
Audit	<code>FinancialTransaction</code> ledger + <code>ActivityLog</code> ; KYC vault states; document-expiry reminders.

2

SEPARATE IDENTITY SYSTEMS

4

JWT TOKEN TYPES

404

RETURNED FOR UNOWNED
DATA